



**Essay / Assignment Title:**

**Eco-Travel Advisor – Conversational Agent for Sustainable Tourism Planning using the Rasa Platform**

**Programme title:**

**Advanced Conversational UI Design and Chatbot Development**

**Name**

**Vidhu**

**Year:**

**2026**

## **Statement of compliance with academic ethics and the avoidance of plagiarism**

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

VIDHU

.....

Date: 01/10/2026

# **CONTENTS**

1. Introduction
  2. Research and Literature Review
  3. System Requirements and Architecture
  4. Conversational Prototype and User Interaction Design
  5. Technical Implementation, AI and NLP
  6. Testing and Evaluation
  7. Deployment
  8. Conclusion
- Reference

# 1. INTRODUCTION

In today's world people are getting more and more independent on their self and most of them likes to plan their trips all over the world by themselves to exactly feel the touch that they are involved in each and every process to make this trip happen, this is totally opposite to the 90's generation as they just want to roam around the world just go to their trip planner and plan their trips but what we do we take the input from both the generation we bring the trip planner to their homes as well as they can connect with them to plan the trips as they want. So the question is how we bring trip advisor to their homes, let deep dive into it and get to know the whole process, what we do we will develop a Rasa backed chatbot named Terra Eco Travel advisor, which you can access on your local machines easily by downloading the package and it helps to plan your trip accordingly as well as people can also choose budget trip to high end trips, less carbon emission trips and their mode of transport. The main objectives of this bot which need to be achieved are given below:-

- Gather travel needs step by step according to the person such as destinations, dates, budgets, and green priorities.
- Search and recommend eco certified hotels, low-carbon transport with help of API..
- Calculate real carbon footprints for different ways of traveling.
- Keep track of the context during a long, multi-turn conversation so the user does not have to repeat themselves.
- Pass the chat to a live human travel agent if the trip becomes too complicated or when the user asks for expert help.

## **2. RESEARCH AND LITERATURE REVIEW**

### **2.1. Sustainable Travel and Conversational AI**

In today's world the new generation know about the climate change as being awareful about the climate, there is the rapid growth of sustainable tourism in all along the world and new generation love to travel by themselves to explore new places and plan their itinerary also because they know that planes, cars, and massive hotels can hurt the planet. However, travelers face a big problem called the "intention-behavior gap." This means many people want to travel eco friendly, but when they actually go to book a trip, they choose the cheapest or easiest option instead because finding eco friendly or less carbon emission choices is too difficult to find. But this gap can be covered by digital travel assistants and chatbots. Instead of making a user look at ten different websites with complicated eco labels, a chatbot lets them just say what they want. It act as interactive recommendation systems. They change the way people get information by breaking down hard environmental data into small, friendly pieces of advice during a chat.

### **2.2. NLP and LLMs in Travel Applications**

NLP stands for Natural Language Processing is the is the foundational technology that helps a computer understand human text. In applications like travel chatbots, NLP used to find the "intent" (what the user wants to do) and "entities" (important details like city names or dates). Also together with NLP, Large Language Models (LLMs) like ChatGPT, Gemini, Claude has become very popular. They can interact with human very easily, but they have some big flaws like sometimes for they can start hallucinating (make up facts) and they can be unpredictable. But for now we use Rasa for our chatbot instead of LLM. Rasa is a rule based model and it gives us full control over the conversation flow which help model understands the user's intent, but strictly, API calls help in live recommendation and human handovers, this all makes sure the bot never makes up fake prices or incorrect carbon numbers.

### **2.3. Conversational UX and Persuasive Design**

The Design of a chatbot is totally different from designing a app or website. While designing a chatbot we also need to take care of user prospective so that they don't get confused in navigating all through the bot so most the chatbot available in market use simple generic design where interaction to bot is the main page where user come and type its prompt and do all the things. Also as a travel chatbot we try to make it simple during the chat so user can interactively with the help of buttons fill all the details instead of big form.

### **2.4. Ethical, Privacy and Accessibility Considerations**

All the Chatbots must follow strict ethical and safety rules:

Data Privacy: Travel data is private and sensitive. It contains personal schedules, locations, and financial plans. The system must follow rules like GDPR and protect personal data carefully.

Accessibility: The chat interface must be simple so that assistive tools like screen readers for visually impaired users can easily read the chat text out loud.

Inclusivity: The system cannot assume everyone has a massive budget or the same high commitment to environmentalism. It must respect cheap budgets and basic eco friendly preferences equally.

## 3. SYSTEM REQUIREMENTS AND ARCHITECTURE

### 3.1. Functional Requirements

Rasa Chatbot system must be able to perform a set of core tasks so that it should not get confused while user is planing the trip:

| Requirement Name           | What the Feature Does in the Chatbot                                                                     |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Trip Intake Module</b>  | Uses a structured Rasa form to ask for destination, origin, dates, budget, and eco-levels step by step.  |
| <b>Location Detection</b>  | Allows users to type a city name or use browser GPS to find their origin coordinates.                    |
| <b>Live Info Retrieval</b> | Connects to external APIs to pull real flights, hotels, and city activities.                             |
| <b>Carbon Calculation</b>  | Sends travel distances to an official environmental database to get carbon emission estimates.           |
| <b>Weighted Scoring</b>    | Scores and ranks choices based on carbon weight, total price, and user priorities.                       |
| <b>Human Escalation</b>    | Packages the entire slot history and transfers it to a human advisor UI.                                 |
| <b>Error Recovery</b>      | Catches gibberish, out-of-order answers, or off-topic rants and brings the user back to the form safely. |

### 3.2. Non-Functional Requirements

Some Non-functional requirements make sure the system runs fast, looks good, and protects privacy of user:

- Usability: The chat language must be plain and simple. The carbon emission scores must show "kg of CO2 emissions" that are being released in easy plain language.
- Latency: The bot must respond in less than three seconds for all basic messages to keep the conversation live and natural.

- Reliability: If an external API crashes, the chatbot should use an offline fallback like mock data to show the recommendations.
- Privacy: All secret software keys must stay hidden in a protected .env file on the server and never be sent to the front end browser.

### 3.3. System Architecture

Backend design of this chatbot uses a modern architecture which is divided into different layers:

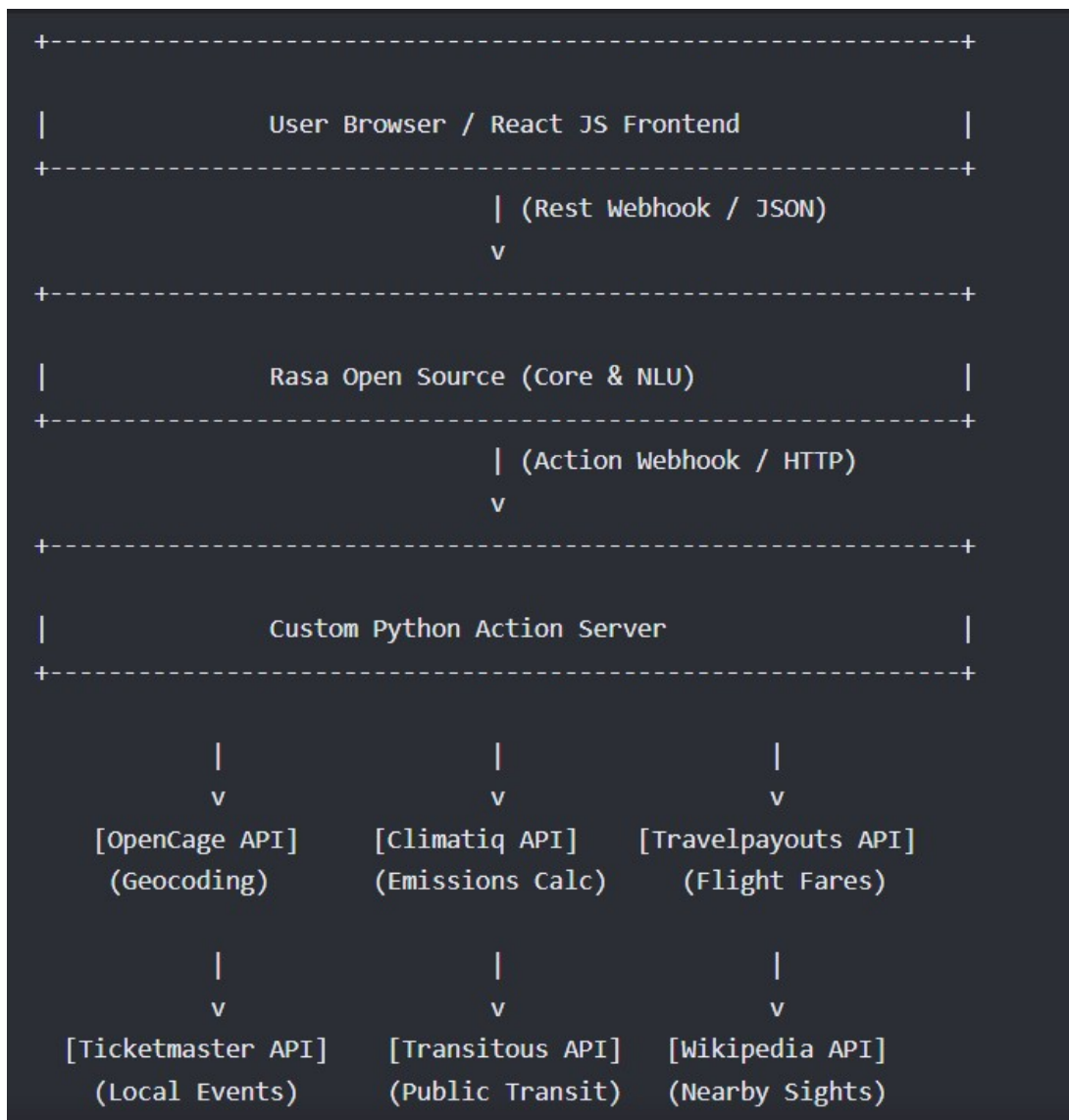


Fig 1

1. Frontend Layer: A web app built with React JS that renders the chat interface and the side data panels.

2. Dialogue & NLP Layer (Rasa): Processes incoming user text messages, classifies intents, extracts entities, and decides the next bot move.
3. Action Server: A Python server that executes custom code inside actions/ files, handles calculations, and runs data queries.
4. External Services Layer (APIs): Live data networks that feed real world data into the chatbot.

### **3.4. Technology and API Selection**

For this Chatbot specific type of free API and tools are selected:

- Rasa Open Source (3.6.21): Chosen because it allows local data hosting and uses explicit rule management.
- Python (3.10.11): The official industry standard language for data science and Rasa actions.
- OpenCage API: Turns plain text city names into exact latitude/longitude map coordinates.
- Travelpayouts API: Finds airport IATA codes and retrieves real, recently observed airline ticket prices.
- Climatiq API: An industry-grade carbon database that provides verified emission numbers for transport routes.
- ORS API:- The Open Route Service (ORS) API is a location-based services platform used to build routing, navigation, and geographic data analysis features into applications

## 4. CONVERSATIONAL PROTOTYPE AND UI/UX

### 4.1. Conversation Flow

Conversation Flow for user is designed in a very simple manner so that user step-by-step plan its trip:

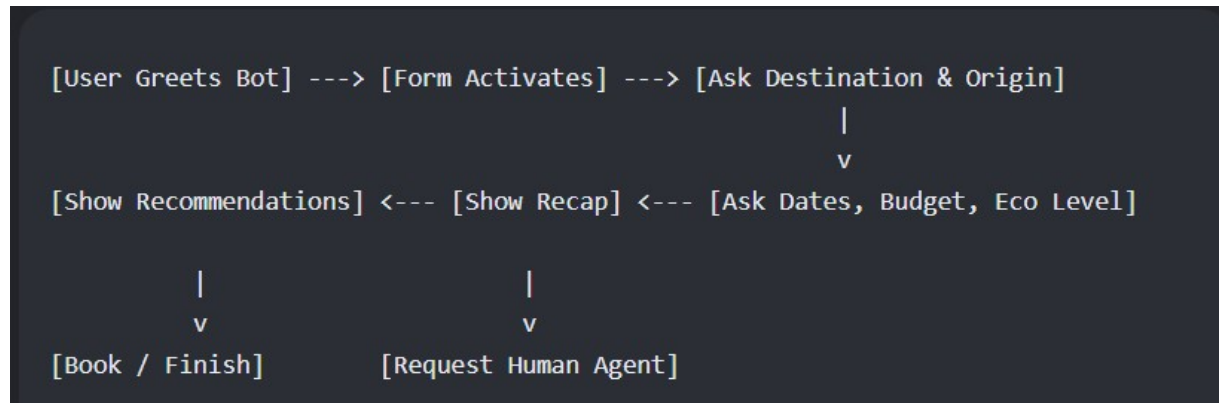


Fig 2

The user starts the bot, and the information form turns on. Chatbot gathers all the required information like location details, travel dates, financial limits, and carbon goals one by one. Once the slots are full, the bot stops and prints a summary card. The traveler checks the data and clicks the button to search for recommendations. The bot runs and collect the information from the API and displays the results results. If user is not able to understand and is getting confused, the user can click "Talk to a specialist" to raise his/her problem to human intervention.

### 4.2. Adaptive Multi-Turn Dialogue

The chatbot uses Rasa slots to remember details across many turns. Slots are like memory buckets. Once a slot is filled, the bot adjusts its questions. For example, if a user opens the chat by typing *"I want to go to Paris next week"*, Rasa's NLU immediately extracts the destination (Paris) and the start date. When the intake form activates, it skips those questions automatically and moves straight to budget and sustainability level. This makes the bot feel intelligent and adaptive rather than repetitive.

### 4.3. User Interface Design

UI design is created by focusing on the chat conversation, by not adding extra layers and confusing menus from the design:

- **Quick Reply Buttons:** These Buttons are used very frequently so that there is less interaction of user typing the prompt so that there is no mistake in understanding the text by the model such buttons are sustainability levels (Standard, Balanced, High, Maximum) or transport preferences.

- **Results Cards:** When travel options are showed to a user, it also shows color status tags on transport cards based on carbon footprints. A green tag means low emissions and red means high carbon impact.
- **Side Panel:** Instead of putting every thing in the chat history, the trip summary shows in a separate docked side panel so that user can check easily that bot is getting all the information correctly or just misrepresenting the data.

#### **4.4. Sustainable Recommendation**

During the time of showing the recommendations, the bot never orders options by price. It takes multiple factors together: distance and carbon metrics and user preferences. If a user selects "Maximum Sustainability," the system prioritizes trains and low-emission routes at the top of the panel, even if they cost more or take more hours than a flight. But in only one case where bot automatically selects flights where trains cannot go but the flights must achieved maximum sustainability in that domain.

#### **4.5. Human Handover and Error Recovery**

Not every chatbot can solve every travel problem perfectly. If a user asks a weird question or gets stuck, the model has different ways to get out of it:

- **Error Recovery:** If the user speaks gibberish or types off-topic text mid-form, the python validates the text and the bot rejects the bad input and repeats the question using helper buttons to understand easily.
- **Human Handover:** If the user keeps failing to get intended recommendations or clicks "Talk to a human," the bot stops and all the chat between the user and bot sent directly to a human advisor.

## **5. TECHNICAL IMPLEMENTATION, AI AND NLP**

### **5.1. NLU Pipeline and Slot Management**

Chatbot system use Rasa DIET Classifier pipeline in config.yml file which add spaCy NLP with the en\_core\_web\_md model for word vectors, which helps model to understand "hotel" and "accommodation" share same meaning by checking their vectors as both words vectors situated near to each other. The text converted to tokens and get its features by running it over 100 epochs. A Fallback Classifier is set at 0.55 confidence to catch the unhelpful messages in the text.

In the chatbot backend system memory slots are defined like destination, origin, and budget in domain.yml. When a user answers the question asked by the bot, validation check runs in the backend in intake\_form.py which check the data. For example, if a destination place like Spain is selected by user, the bot sent it to OpenCage for geocoding, if it get verified, the name is saved in the file, if not the slot of place resets to none, so that bot is able to ask again the place name.

### **5.2. Dialogue Logic and External APIs**

In chatbot, Memoization and Rule policy is define in rules.yml for dialogue logic, if user start the chat with "I want to go to Paris", the bot fills the slot instantly and skip the question during intake form.

The backend actions are connected to live data APIs:

- Climatiq API: Climatiq API calculate the carbon emissions in kilograms.
- Travel payouts: Travel payout API tracks live airfares, train and bus where ever its possible to go with any mode of transport.
- Overpass & Wikipedia: Check hotels from Open Street Map and local landmarks using destination coordinates.

### **5.3. Ranking, Fallbacks, and Handover**

Action Rank Recommendation calculates total trip distance with Haversine formula. If trip distance is greater than 1,000 km, it only choose the flight mode as the other travel modes are not possible. All the routes ranking by any mode of transport is decided by its carbon impact, price, and dates.

If the user sends gibberish text, the bot increase the hidden confusion\_count slot and set to show quick reply buttons for better understanding. If the user failed second time, the bot stops the form and tries to give option to transfer the chat to the human advisor with full context of the chat with the user.

## 6. TESTING AND EVALUATION

### 6.1. NLU Testing and Validation

Before proceeding ahead we need to check and test our chatbot how well our chatbot works and understand the text features so for that we ran a 3 fold cross validation test by using the following command in the command prompt:

```
rasa test nlu --cross-validation --folds 3
```

Overall this model achieved intent accuracy of total 60.86% which is good in the start. The evaluation report show that some intents are still hard for the model to to understand. For example, confirm\_positive scored a low F1 score of 0.34 because of short words like "that works" or "yep" are easily confused with confirm\_negative. Also, provide\_accommodation\_preference and provide\_sustainability often mix together because word like "eco" which fit into both intents.

In entities, the system got a micro-average F1-score of 0.61. The main problem is between origin and destination as they both overlap each other. The confusion matrix shows that 18 destination words are labeled as origins and 9 origin words are labeled as destinations because city names look exactly the same without clear context sentences also one more confusion happen with travel dates, where travel\_start had a 0.00 score because all 35 examples were labeled as travel\_end. All this happens because the model struggle to know the difference between start and end dates when these two parameters looks same.

### 6.2. Dialogue and Story Testing

Now check the conversation flow logic using test stories by using the following command:

```
rasa test core --stories tests/test_stories.yml
```

The test file in test\_stories.yml has 20 different conversation dialogues. These tests check how the bot handle unique opening messages, flexible schedule, and different budget.

Result shows chatbot passed all the dialogue testing with 100% passing rate (20/20 stories passed). This test proved that if a user types random gibberish like "xyzyy plugh wibble" in between the conversation, the system bypass it easily and don't let the bot confused. It keeps the user inside the active loop instead of breaking the entire chat.

### 6.3. Live API and User Experience Evaluation

Also we test custom python scripts using a mock tool to simulate internet problems when external servers like ClimaTiq return a 403 error, the code safely catches it and shifts to offline math constants so the chatbot never crashes. And looking at database chat logs, it shows that users dislike too many text bubbles. That's why we changed the action server to run silently in

the background. Now, the bot sends only two easy bubbles: a quick confirmation message and one simple results deck.

## 7. DEPLOYMENT

### 7.1. Deployment Architecture Options

Chatbot is designed to deploy in two different ways as user requirement or where the user want to run it one is directly on command prompt and power shell using bash scripts for rasa and other one is docker container. When using Docker, system supports two modes. Local Container Mode uses Docker Compose to divide the chatbot system into four separate services (rasa, actions, admin, and frontend) running on a shared virtual network(VN). Other one supports cloud hosting on which Hugging Face Spaces and everything is combined into a single container image managed by start.sh script and fronted managed by Nginx reverse proxy on one public port (7860).

Also if the user want to test and run the chatbot locally through Command Prompt. The Command Prompt need four adjacent terminal windows at the same time to start the bot on the local machine and run it on the local server:

- Terminal 1 (Action Server): Runs the command `rasa run actions --port 5055` to start the Python code that enables APIs to collect and provide live data to the chatbot.
- Terminal 2 (Rasa Server): Start the terminal using `rasa run --enable-api --cors "*" --port 5005` to launch the NLU engine so the React frontend can interact with the chatbot.
- Terminal 3 (Admin Panel): To run the Admin server site, run `admin_server.py` on command prompt. But this terminal is not necessary only if you want to visit the admin site also you need credentials to enter the admin site.
- Terminal 4 (Frontend): It runs the frontend of the chatbot by using `npm run dev` command. But NPM must be installed in your local system.

Note:- Download all the requirements before starting the terminal and start the virtual environment for rasa by running launching `.venv\Scripts\activate.bat` and then your are good to go.

### 7.2. Security and Live Limitations

The security of the chatbot is managed strictly across both deployment parts. The `.dockerignore` file keeps your private credential `.env` files out of the container layers. The admin server checks origins of the IP by (`ADMIN_ALLOWED_ORIGINS`) to check the IP is malicious or not, if it is malicious then it will block that web traffic. There is one limitation in the bot only when running through docker containers which is Cold Start 502 delay on Hugging Face, where Nginx has to wait for few seconds for the machine learning models to finish loading during bootup.

## **CONCLUSION**

Overall the Terra Eco-Travel Advisor chatbot successfully shows how a conversational framework can help people plan there Eco friendly trips with ease. By using the Rasa open source platform, the system suggest travelers intended recommendation with all the eco tags, as well as collect the information from the user using interactive buttons and suggestion like budgets and distances. The recommendation generated by chatbot comes through multiple distinct external APIs (such as Travelpayouts, Ticketmaster, and OpenCage).

## References

- Tera bot github repo:- <https://github.com/VidhuGupta/Eco-Travel-Advisor-Conversational-Agent-for-Sustainable-Tourism-Planning-using-the-Rasa-Platform>
- Duffel-API:- <https://duffel.com/>
- ClimaTiq-API:- <https://www.climatiq.io/docs>
- Travel Payouts-API:- <https://support.travelpayouts.com/hc/en-us/sections/360004490151-API-by-Travelpayouts>
- Open Cage-API:- <https://opencagedata.com/api>
- ORS-API:- <https://api.openrouteservice.org/>
- Rasa:- <https://rasa.com/docs>
- Docker:- <https://docs.docker.com/>
- React JS:- <https://react.dev/>
- AI like chatgpt, claude, gemini is used to write intents, actions, stories, defining rules, frontend of the chatbot and also litte help in formating this report.